Contents

ISO 13485:2016 Readiness Plan — using `speccheck` for medical device software 1

1. Framing . . . 1
2. What already works in our favour . . . 2
3. Work items, roughly by priority . . . 2
   - 3.1 Validation pack (documents, mostly no code) . . . 2
   - 3.2 Record integrity (code changes) . . . 2
   - 3.3 Risk traceability (a spec change) . . . 2
   - 3.4 The LLM judge in regulated use (a policy decision) . . . 3
   - 3.5 Our own development process . . . 3
   - 3.6 Templates customers can map to their own procedures . . . 3
4. Suggested order . . . 3
5. Next actions . . . 3

ISO 13485:2016 Readiness Plan — using `speccheck` for medical device software

> **Status:** draft plan, not a spec change · **Date:** 2026-11-05 · **Applies to:** `speccheck` v1.24 (code 1.24.0) **Goal:** make `speccheck` usable by a medical device manufacturer inside an ISO 13485:2016 QMS. **Caveat:** before making compliance claims to customers, have a regulatory/QA consultant review this plan. Auditors differ on how much tool validation they expect.

1. Framing

`speccheck` cannot comply with ISO 13485 by itself, because the standard certifies an organization's quality management system, not a tool. The real goal is for a manufacturer to be able to use `speccheck` inside an ISO 13485 QMS. In that role it is **software used in the quality management system**, and these clauses apply:

Clause	Obligation	Relevance to `speccheck`
4.1.6	Validate QMS software for its intended use before first use and after changes, in proportion to its risk	The tool needs a validation pack (§3.1)
4.2.4 / 4.2.5	Control of documents and records	SPEC.md is a controlled document; the reports are records (§3.2)
7.3.2 / 7.3.3	Design inputs and outputs	SPEC.md R/C/I/K/E rows are design inputs
7.3.6 / 7.3.7	Design verification and validation	The conformance report is verification evidence

An auditor will also read the tool against these companion standards and regulations:

- **IEC 62304**: software lifecycle, traceability, safety classes, SOUP (third-party components)
- **ISO 14971**: risk management
- **IEC 82304-1**: health software products
- **FDA QMSR** (21 CFR 820, which incorporates ISO 13485)
- **21 CFR Part 11**: electronic records and signatures
- **FDA Computer Software Assurance** guidance
- **ISO/TR 80002-2**: validation of software used in a medical device quality system

2. What already works in our favour

- **The core is deterministic.** Byte-identical output from the same inputs is about the best property a QMS tool can have. You validate it once and it reproduces.
- **It fails safe.** The LLM judge can only downgrade a verdict, never upgrade one. That is a risk control you can name explicitly in the tool risk assessment.
- **It checks itself.** `speccheck` verifies its own SPEC.md: v1.24 reports CONFORMING 307/307, there is a Lean proof (`proof/`, `proof_from_spec/`), and `uv.lock` pins dependencies. That is most of an operational-qualification evidence base.
- **It already does requirement→test traceability** (§11 of SPEC.md and the per-ID evidence in the report), which IEC 62304 §5.7 and ISO 13485 §7.3 both require.

3. Work items, roughly by priority

3.1 Validation pack (documents, mostly no code)

This is what customers will actually ask for.

- ☐ **Intended-use statement and scope boundary**, for example: "the deterministic `check` with `--judge none/mock` is the validated function; the LLM judge is advisory and outside validated use."
- ☐ **Tool risk assessment.** The hazardous failure is a **false PASSING**: an ID reported as verified when it isn't. List each cause and the control that covers it:
  - citation-grep false positive → T-ids covering the C-01/C-03 grammar
  - JUnit misattribution → C-04 tests
  - a stale or unrelated result file → `--strict`, the provenance manifest (§3.2)
  - an algorithmic error in status computation → the Lean proof of C-05
- ☐ **IQ/OQ protocol** that the customer runs in their own environment: a script built from the existing suite and fixtures, plus a signed report template.
- ☐ **Release notes per version** and a **known-anomalies list**.
- ☐ **Change-control statement**: which spec changes force revalidation (for example, any change to C-05, the JSON schema version, or the citation grammar).

3.2 Record integrity (code changes)

The report is a QMS record, so it has to show what it was computed from. Right now `speccheck.json` deliberately carries no timestamps or versions (C-07), and the only hash it records is `judge_prompt_sha256`.

- ☐ Add a separate **provenance manifest** and keep the main reports byte-identical. The manifest holds:
  - the tool version
  - SHA-256 hashes of SPEC.md, the JUnit file, the proof manifest, and the source tree (or its git commit)
  - the time and the operator
- ☐ Optionally, add a `verify` command that re-checks a report against its manifest.
- ☐ State in the intended use that signatures, approvals, and audit trails (Part 11) are delegated to the customer's eQMS or git.

3.3 Risk traceability (a spec change)

IEC 62304 and ISO 14971 need traceability from hazard to risk control to requirement to verification.

- ☐ Choose one of these options:
  - **Option A:** add a hazard family (e.g. `H-nn`) with a `mitigated_by` edge, so the report can show every risk control verified.
  - **Option B:** support a foreign namespace (v1.23 C-22) so IDs from the customer's risk file can be cited.
- ☐ Add a way to tag requirements with an IEC 62304 **software safety class** (A/B/C), so that `--strict` can be stricter for class C (for example, no `EXECUTES_ONLY` allowed, and `UNPROVEN` treated as failing).

3.4 The LLM judge in regulated use (a policy decision)

The judge is non-deterministic, depends on the network, and the model can drift. Choose one:

- ☐ **Exclude** it from validated use entirely (simplest), or
- ☐ **Control** it:
  - require a pinned model ID
  - store each request and response as a record
  - treat the T-49 accuracy runs as ongoing performance qualification, revalidated whenever the model or `judge_prompt_sha256` changes

3.5 Our own development process

You don't need ISO 13485 certification to sell the tool. But customers doing supplier qualification will look for a documented lifecycle, version control, defect handling, and a release process.

- ☐ Write a short **development process document**. The spec-proposal → spec-review → spec-plan → spec-build → spec-proof workflow already produces most of the evidence; it just needs writing up.

3.6 Templates customers can map to their own procedures

- ☐ A SPEC.md profile and report layouts that map directly onto an IEC 62304 Software Requirements Specification, traceability matrix, and verification report.
- ☐ Sections for **SOUP** (third-party components) and a **safety-class header**.

4. Suggested order

Step	Item	Kind	Rationale
1	3.1 Validation pack	documents	Makes the deterministic core usable in a QMS today
2	3.2 Provenance manifest	code	Makes the reports usable as QMS records
3	3.4 Judge policy	decision	Cheap to decide, and it sets the validation scope
4	3.3 Risk traceability	spec change	The biggest change; go through `spec-proposal`
5	3.5, 3.6	ongoing	Supplier qualification and adoption

5. Next actions

- Write 3.2 and 3.3 up as a `spec-proposal` (draft rows plus a decision table), targeting v1.25.
- Draft the intended-use statement and the tool risk assessment (3.1).